

Intermediate (feel the speed difference) — Solution

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Story — `gather_practice.py`: reading "concurrent is faster than sequential" is not the same as timing 6 seconds against 2 seconds yourself, on the same 3 tasks — this is the exercise that turns that claim into something you measured. **If not:** every later use of `asyncio.gather` in this curriculum would rest on a speedup you took on faith instead of one you watched happen.

Basic Version

Approach 1 — the direct way

```
# gather_practice.py – Intermediate section
import asyncio
import time

async def f1():
    await asyncio.sleep(2)
    return "f1 done"

async def f2():
    await asyncio.sleep(2)
    return "f2 done"

async def f3():
    await asyncio.sleep(2)
    return "f3 done"

async def run_sequential():
    start = time.perf_counter()
    await f1()
    await f2()
    await f3()
    print("sequential:", time.perf_counter() - start)

async def run_parallel():
    start = time.perf_counter()
```

```
await asyncio.gather(f1(), f2(), f3())
print("parallel:", time.perf_counter() - start)

asyncio.run(run_sequential())
asyncio.run(run_parallel())
```

Expected output (your exact numbers will vary slightly):

```
sequential: 6.01
parallel: 2.01
```

`sequential` prints something close to **6 seconds** — $2 + 2 + 2$, because each `await` waits for the previous task to fully finish before the next one even starts. `parallel` prints something close to **2 seconds** — all three `asyncio.sleep(2)` calls run at the same time, so the total is roughly the length of one task, not three. That gap is the whole point of this exercise.

This version works correctly. It's missing type hints and an `if __name__ == "__main__":` guard — both fine for a first working version.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Approach 1 — typed, with a `__main__` guard

```
# gather_practice.py – Intermediate section
import asyncio
import time

async def f1() -> str:
    await asyncio.sleep(2)
    return "f1 done"

async def f2() -> str:
    await asyncio.sleep(2)
    return "f2 done"

async def f3() -> str:
    await asyncio.sleep(2)
    return "f3 done"

async def run_sequential() -> None:
    start = time.perf_counter()
    await f1()
    await f2()
    await f3()
    print("sequential:", time.perf_counter() - start)

async def run_parallel() -> None:
    start = time.perf_counter()
    await asyncio.gather(f1(), f2(), f3())
    print("parallel:", time.perf_counter() - start)

if __name__ == "__main__":
```

```

asyncio.run(run_sequential())
asyncio.run(run_parallel())

```

Expected output:

```

sequential: 6.01
parallel: 2.01

```

Difference from Basic: full type hints (`-> str` on the task functions, `-> None` on the runners) make the contract of each function readable without opening its body. The `if __name__ == "__main__":` guard means this file's functions can be imported and reused (say, from a test) without immediately firing off 8 seconds of sleeping on import. Same two numbers come out either way — roughly 6 seconds for `sequential`, roughly 2 for `parallel`.

Approach 2 — one task fails, and it costs you every result

Story: these examples so far all assume every task succeeds — the realistic case, once these aren't `sleep()` calls but real network requests, is that one of them doesn't. **If not:** the first real failure in a `gather()` call would surprise you by silently discarding 2 good results along with the 1 bad one.

```

# gather_practice.py – Intermediate section
import asyncio

async def f1() -> str:
    await asyncio.sleep(2)
    return "f1 done"

async def f2_broken() -> str:
    await asyncio.sleep(2)
    raise ValueError("f2 failed on purpose")

async def f3() -> str:
    await asyncio.sleep(2)
    return "f3 done"

async def run_parallel_unsafe() -> None:
    results = await asyncio.gather(f1(), f2_broken(), f3())
    print(results)

```

```
if __name__ == "__main__":
    asyncio.run(run_parallel_unsafe())
```

Expected output:

```
Traceback (most recent call last):
...
ValueError: f2 failed on purpose
```

`f1` and `f3` both actually finished successfully — but you never get to see either result, because `gather` re-raises `f2`'s exception the moment everything has settled, and the whole call just raises instead of returning a list.

Approach 3 — `return_exceptions=True`, so one failure doesn't cost the other two

Story: the default behavior above throws away 2 good results the instant a 3rd, unrelated task fails — fine when the tasks genuinely depend on each other, a real bug when they don't. **If not:** one failed tool call in a multi-tool agent turn would silently wipe out every other tool's result that already succeeded.

```
# gather_practice.py – Intermediate section
import asyncio

async def f1() -> str:
    await asyncio.sleep(2)
    return "f1 done"

async def f2_broken() -> str:
    await asyncio.sleep(2)
    raise ValueError("f2 failed on purpose")

async def f3() -> str:
    await asyncio.sleep(2)
    return "f3 done"

async def run_parallel_safe() -> None:
    results = await asyncio.gather(
        f1(), f2_broken(), f3(), return_exceptions=True,
    )
    for result in results:
        if isinstance(result, Exception):
```

```

        print("a task failed:", result)
    else:
        print("a task succeeded:", result)

if __name__ == "__main__":
    asyncio.run(run_parallel_safe())

```

Expected output:

```

a task succeeded: f1 done
a task failed: f2 failed on purpose
a task succeeded: f3 done

```

`return_exceptions=True` changes what `gather` gives back: a list, always, the same length as the number of tasks, in the same order — where a failed task's slot holds the exception object itself instead of raising. `isinstance(result, Exception)` is what tells a real success apart from a captured failure once they're sitting in the same list.

Difference from Approach 1, and between Approaches 2/3: Approach 1 never tests what happens when a task fails — every example assumes all 3 succeed. Approach 2 shows the default, and often surprising, behavior: 2 successful results, thrown away, because a sibling task failed. Approach 3 is the fix — one added keyword argument that turns "one bad call wipes out everything" into "every result comes back, labeled."

Which one should you actually write? Default `gather()` (no `return_exceptions`) is the right choice when any one failure genuinely should stop the whole group — if `f2` failing means the other 2 results are now useless anyway, let it raise and handle it with a normal `try/except` around the `gather` call. Reach for `return_exceptions=True` the moment the 3 tasks are genuinely independent and you want whatever succeeded even if something else didn't — 3 separate tool calls for one agent turn is exactly that case: a failed web search shouldn't throw away a calculator result that already came back fine.